

SalesIQ

Intelligent Sales Analytics & Forecasting Platform

AI/ML Internship Assignment

Project Specification & Implementation Guide

Recommended duration: 5–7 days

Difficulty: Intermediate–Advanced

Confidential — Internship Evaluation Material

1. Project Overview

SalesIQ is an AI/ML-powered sales analytics and forecasting platform designed to help a business understand historical sales performance and estimate future demand and revenue. The intern will build a complete pipeline covering data ingestion, preprocessing, exploratory analysis, feature engineering, forecasting, model evaluation, REST APIs, and an interactive dashboard.

The project should demonstrate practical machine-learning engineering rather than simply training a model. The final system should make it possible for a user to upload sales data, inspect trends, select a forecast horizon, view predictions, compare models, and understand the factors or patterns behind the forecast.

2. Problem Statement

Businesses often have large amounts of historical sales data but lack a simple way to convert that data into reliable forward-looking information. Manual spreadsheets can reveal what happened, but they are less effective for identifying seasonality, trends, demand changes, and expected future sales.

Build SalesIQ to address this problem by creating an end-to-end forecasting and analytics system that takes historical transaction/sales data and produces actionable forecasts and visual insights.

3. Project Objectives

- Build a clean and reusable sales-data ingestion and preprocessing pipeline.
- Perform exploratory data analysis to identify trends, seasonality, outliers, and product/region performance.
- Engineer meaningful time-based and business features.
- Implement and compare at least two forecasting approaches.
- Evaluate models using appropriate time-series validation and forecasting metrics.
- Expose the forecasting pipeline through a REST API.
- Create an interactive dashboard for analytics and forecasts.
- Document assumptions, methodology, model limitations, and setup instructions.
- Package the application so another developer can run it locally.

4. Target Users

User	Primary Need
Business / Sales Manager	Understand performance and expected future sales.
Business Analyst	Explore trends, products, regions, and forecast scenarios.
Operations / Inventory Team	Use demand forecasts for planning.
Technical Team	Run, evaluate, and maintain the ML pipeline.

5. Core Features

Feature	Expected Behavior
Data Upload	Upload CSV sales data and validate required columns.
Data Validation	Detect missing fields, invalid dates, duplicate records, and invalid numeric values.
Data Cleaning	Handle missing values, duplicates, inconsistent formats, and obvious data-quality issues.
EDA Dashboard	Show sales trends, growth, product performance, regional performance, and distributions.
Feature Engineering	Create date, lag, rolling, trend, and other relevant features.

Forecasting	Generate forecasts for selectable horizons such as 7, 30, or 90 days.
Model Comparison	Compare multiple models using consistent validation and metrics.
Prediction Intervals	Where supported, show a forecast range rather than only a point estimate.
Product/Region Forecasting	Optional advanced forecasting at product or regional level.
Anomaly Detection	Optional identification of unusual sales periods.
API Layer	Expose data, analytics, training/evaluation, and forecasting operations through REST endpoints.
Interactive Dashboard	Provide filters, charts, forecast views, and key metrics.
Export	Optional export of forecast results as CSV.

6. Dataset Specification

The recommended dataset should contain at least 6–24 months of historical data. A synthetic dataset may be provided to the intern.

Column	Type	Description	Required
date	Date	Transaction or sales date.	Yes
product_id	String	Unique product identifier.	Yes
product_name	String	Product name.	Yes
category	String	Product category.	Recommended
region	String	Sales region.	Recommended
units_sold	Numeric	Number of units sold.	Yes
unit_price	Numeric	Selling price per unit.	Recommended
discount	Numeric	Discount applied.	Optional
marketing_spend	Numeric	Marketing expenditure for the period.	Optional
revenue	Numeric	Revenue generated.	Yes

Example record:

2026-01-15 | P001 | Laptop Pro | Electronics | North | 12 | 60000 | 5 | 25000 | 684000

7. Machine Learning / Forecasting Requirements

7.1 Baseline

Implement a simple baseline such as a moving average or seasonal-naive forecast. The baseline provides a reference point against which more sophisticated models must be evaluated.

7.2 Forecasting Models

Implement at least two forecasting approaches. Suitable choices include:

- ARIMA / SARIMA using statsmodels.
- XGBoost or another tree-based regression model with lag and calendar features.
- Prophet, if appropriate to the dataset and environment.
- LSTM/GRU as an optional advanced model; it is not required simply for complexity.

7.3 Feature Engineering

- Day, week, month, quarter, year, and day-of-week.
- Lagged sales values such as lag-1, lag-7, lag-14, and lag-30 where appropriate.
- Rolling mean and rolling standard deviation.

- Recent growth rates.
- Product/category/region aggregates.
- Discount and marketing features when available.

7.4 Validation

Do not randomly shuffle time-series observations. Use a chronological train/validation/test strategy or rolling/expanding-window validation so that future information does not leak into training.

7.5 Evaluation Metrics

- MAE — Mean Absolute Error.
- RMSE — Root Mean Squared Error.
- MAPE — Mean Absolute Percentage Error, with appropriate handling of zero/near-zero actuals.
- R^2 may be reported for regression-style models but should not replace forecasting-specific metrics.

The intern must provide a model-comparison table and explain why the final model was selected. Accuracy alone is not sufficient.

8. Proposed System Architecture

A reference architecture is shown below. The intern may modify the architecture if they can justify the decision.

- Frontend (React/Next.js) → Dashboard, upload, filters, forecast visualization.
- FastAPI backend → REST endpoints, validation, orchestration, and ML service integration.
- ML pipeline → preprocessing, feature engineering, training, validation, forecasting.
- Database → PostgreSQL for datasets, forecast results, model metadata, and application data.
- File storage → Local storage for development; object storage may be used as an optional enhancement.
- Docker → Reproducible development and deployment environment.

9. Recommended Tech Stack

Layer	Recommended Technology	Purpose
Language	Python 3.11+	ML pipeline and backend.
Data	Pandas, NumPy	Cleaning, transformation, analysis.
ML	Scikit-learn	Preprocessing and ML utilities.
Time Series	Statsmodels	ARIMA/SARIMA and statistical forecasting.
Optional ML	XGBoost	Feature-based forecasting.
Backend	FastAPI	REST API.
Database	PostgreSQL	Persistent application data.
Frontend	React / Next.js	Dashboard.
Charts	Recharts / Plotly	Interactive visualization.
Styling	Tailwind CSS	UI styling.
Containerization	Docker	Reproducible runtime.
Version Control	Git + GitHub	Source control and submission.

10. Suggested REST API

Method	Endpoint	Purpose
POST	/api/datasets/upload	Upload and validate a CSV dataset.
GET	/api/datasets	List uploaded datasets.
GET	/api/analytics/summary	Return key sales KPIs.
GET	/api/analytics/trends	Return historical trend data.

POST	/api/models/train	Train selected forecasting model(s).
GET	/api/models	List available/trained models.
POST	/api/forecast	Generate a forecast for a selected horizon.
GET	/api/forecast/history	Retrieve previous forecast results.
GET	/api/health	Service health check.

11. Dashboard Requirements

- KPI cards: total revenue, units sold, average order/sales value, growth rate.
- Historical sales trend chart.
- Forecast chart with historical and future periods clearly distinguished.
- Forecast horizon selector: 7/30/90 days.
- Product/category/region filters.
- Top-performing products or categories.
- Regional performance.
- Model comparison section.
- Forecast error/metric section.
- Data-quality indicators showing missing or invalid records.

12. Advanced / Bonus Features

- Product-level and region-level forecasting.
- Automatic model selection based on validation performance.
- Prediction intervals / uncertainty visualization.
- Anomaly detection for unusual sales periods.
- What-if analysis for discount or marketing-spend scenarios.
- Scheduled model retraining.
- Background training jobs using Celery/RQ or a task queue.
- Authentication and role-based access.
- CSV export of forecasts and analytics.
- Docker Compose for frontend + backend + database.
- Cloud deployment.
- ML experiment tracking using MLflow.
- Unit and integration tests.

13. Expected Deliverables

- Complete source-code repository.
- Working frontend dashboard.
- Working FastAPI backend.
- ML training and forecasting pipeline.
- Sample dataset.
- requirements.txt or pyproject.toml.
- Dockerfile and, preferably, docker-compose.yml.
- README.md with architecture, setup, API documentation, and usage.
- Model evaluation report with comparison of models.
- Screenshots or short demo video.

- A short technical document explaining assumptions, preprocessing, features, validation, results, and limitations.

14. Suggested Repository Structure

```
salesiq/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── services/
│   │   ├── models/
│   │   └── main.py
│   ├── ml/
│   │   ├── preprocessing/
│   │   ├── features/
│   │   ├── models/
│   │   ├── evaluation/
│   │   └── forecasting/
│   ├── tests/
│   └── requirements.txt
├── frontend/
│   ├── src/
│   └── package.json
├── data/
│   └── sample_sales.csv
├── notebooks/
├── Dockerfile
├── docker-compose.yml
└── README.md
```

15. Acceptance Criteria

- Application starts successfully using documented instructions.
- CSV upload and validation work with the supplied dataset.
- Data cleaning handles the documented edge cases.
- At least two forecasting models plus a baseline are implemented.
- Chronological/time-aware validation is used.
- Forecasting metrics are calculated correctly and reported.
- A forecast can be generated for at least one future horizon.
- Dashboard displays historical performance and forecast results.
- API endpoints are documented and functional.
- No obvious data leakage is present.
- Code is reasonably modular, readable, and documented.
- README contains setup, architecture, usage, assumptions, and limitations.

16. Edge Cases to Handle

- Missing dates or invalid date formats.
- Duplicate rows.
- Missing sales/revenue values.
- Zero sales periods.
- Negative or invalid numeric values.
- Products with very little historical data.
- New products with no historical observations.

- Large spikes or drops in sales.
- Gaps in the time series.
- Zero/near-zero values when calculating percentage errors.

17. Internship Evaluation Rubric

Area	Weight	What Is Evaluated
Data preprocessing & quality	15%	Validation, cleaning, missing values, duplicates, edge cases.
EDA & business analysis	10%	Useful insights and appropriate visualizations.
ML / forecasting	25%	Model selection, features, implementation, forecasting quality.
Validation & evaluation	15%	Time-aware validation and correct metrics.
Backend/API	10%	API design, error handling, modularity.
Frontend/dashboard	10%	Usability, clarity, visualizations.
Code quality	5%	Structure, readability, maintainability.
Documentation	5%	README and technical explanation.
Bonus/innovation	5%	Useful advanced features and engineering decisions.

18. What the Intern Should Explain

- Why was the selected forecasting approach appropriate for the dataset?
- What patterns were found during EDA?
- Which features were created and why?
- How was data leakage prevented?
- How was the validation strategy selected?
- Why did one model perform better or worse than another?
- What are the limitations of the forecast?
- How would the system be improved for production-scale data?

19. Recommended 7-Day Work Plan

Day	Focus	Expected Output
Day 1	Requirements, dataset, project setup	Repository + dataset + basic API/UI skeleton.
Day 2	Cleaning + EDA	Clean pipeline + initial analytics.
Day 3	Feature engineering + baseline	Feature pipeline + baseline forecast.
Day 4	Model implementation	At least two forecasting models.
Day 5	Validation + evaluation	Metrics + model comparison + final model.
Day 6	API + dashboard integration	End-to-end working application.
Day 7	Testing + documentation + polish	Final submission + README + demo.

20. Important ML Considerations

Forecasts are estimates, not guarantees. The intern should document assumptions and limitations, including changes in market conditions, promotions, product launches, stockouts, holidays, missing data, and structural changes that may not be represented in historical data.

A sophisticated model is not automatically a better model. The evaluation should emphasize sound validation, data quality, reproducibility, interpretability, and whether the model improves over a sensible baseline.

21. Final Submission Checklist

- ☐ Source code pushed to the assigned Git repository.
- ☐ README completed.
- ☐ Dataset/sample data included or clearly documented.
- ☐ Backend API runs successfully.
- ☐ Frontend dashboard runs successfully.
- ☐ Forecasting models implemented.
- ☐ Model comparison included.
- ☐ Evaluation metrics reported.
- ☐ Tests included for important logic.
- ☐ Demo/screenshots provided.
- ☐ Known limitations documented.

22. Final Outcome

The completed SalesIQ application should demonstrate an end-to-end AI/ML workflow: raw sales data → validation and cleaning → exploratory analysis → feature engineering → forecasting → model evaluation → REST API → interactive dashboard. The emphasis is on sound engineering and reasoning, not merely achieving a high numerical score.